

API Design Patterns

Nhóm 10

Tổng quan các **Design Pattern**



1. CRUD Pattern Mẫu thiết kế cơ bản nhất, tương tác trực tiếp với tài nguyên qua các phương thức HTTP tiêu chuẩn.



2. Query Pattern Kiểm soát và tối ưu hóa luồng dữ liệu lớn thông qua tham số (Phân trang, Lọc, Sắp xếp).



3. HATEOAS Pattern API "tự dẫn đường" bằng cách cung cấp các liên kết để thực thi hành động tiếp theo.

Tổng quan các Design Pattern



4. Event-Driven Pattern Kiến trúc hướng sự kiện, phát triển từ Observer design pattern, dễ dàng mở rộng và phát triển



5. Webhook Pattern Mô hình gọi ngược (Reverse API), Server tự động đẩy dữ liệu về Client thay vì chờ Client truy vấn.

1. CRUD Pattern

Ánh xạ trực tiếp các thao tác từ cơ sở dữ liệu vào các chuẩn **HTTP Methods**. Mẫu này cực kỳ hiệu quả cho các dịch vụ quản lý tài nguyên.

- **POST**: Tạo mới tài nguyên.
- **GET**: Truy xuất dữ liệu.
- **PUT/PATCH**: Cập nhật toàn bộ hoặc 1 phần.
- **DELETE**: Xóa tài nguyên.

```
@app.route("/books/<int:book_id>", methods=["GET"])
def get_book(book_id):
    book = find_book(book_id)
    if not book:
        return jsonify({"error": "Book not found"}), 404
    return jsonify(book), 200
```

2. Query Pattern

Khi CSDL chứa hàng triệu bản ghi, việc GET /users sẽ gây quá tải. Query Pattern sử dụng **URL Parameters** để điều khiển kết quả trả về.

→ **Pagination:** ?page=1&limit=20

→ **Filtering:** ?genre=Sci-Fi

→ **Sorting:** ?order=desc

Kết quả trả về phải bọc trong object data và cung cấp thông tin phân trang để client xử lý

```
@app.route("/books", methods=["GET"])
def get_books():
    q = request.args.get("q", "").lower()
    author = request.args.get("author", "").lower()
    genre = request.args.get("genre", "").lower()
    year = request.args.get("year")
    sort_by = request.args.get("sort_by", "id")
    order = request.args.get("order", "asc").lower()
    page = request.args.get("page", 1, type=int)
    per_page = request.args.get("per_page", 10,
                                type=int)
```

3. HATEOAS Pattern

Hypermedia As The Engine Of Application State

Xây dựng API "thông minh" bằng cách đính kèm các đường link gợi ý hành động ngay trong Response.

- Client không cần thiết ghi nhớ toàn bộ các URLs.
Tính tự tài liệu hóa cao.
- **Bảo vệ logic nghiệp vụ:** Nếu user không có quyền, API đơn giản là không trả về link đó và UI sẽ tự động ẩn nút.

```
@app.route("/books/<int:book_id>", methods=[ "PUT" ])
def update_book(book_id):
    book = find_book(book_id)
    if not book:
        return jsonify({"error": "Book not found",
"links": [{"rel": "self", "href": "/books",
"method": "GET"}]}), 404

    data = request.get_json()
    if not data:
        return jsonify({"error": "Request body is
required", "links": book_links(book)}), 400
```

4. Event-Driven Pattern

Dựa trên Observer design pattern. Tránh tình trạng thắt cổ chai (Bottleneck) khi controller phải làm quá nhiều việc với mỗi một request.

- **Publisher:** Dịch vụ phát sinh dữ liệu chỉ cần tạo ra sự kiện.
- **Subscriber:** Các dịch vụ quan tâm tự động lắng nghe và xử lý độc lập. Subscriber nhận được sự kiện sẽ thực thi bất đồng bộ với luồng chính

Tăng cường độ chịu lỗi và tính tách biệt (Loosely coupled) của hệ thống. Khi một thành phần hỏng thì các thành phần khác vẫn có thể tiếp tục hoạt động

```
@app.route("/books", methods=["POST"])
def create_book():
    ...
    bus.publish("book.created", book)
    return jsonify(book), 201

class EventBus:
    def publish(self, event_type, data):
        for handler in
self._subscribers[event_type]:
            Thread(target=handler, args=(event_type,
data), daemon=True).start()
```

5. Webhook Pattern

Mô hình gọi ngược. Thay vì ứng dụng của ta liên tục gọi API đi hỏi đối tác "Xong chưa?" (Polling) gây lãng phí tài nguyên, server sẽ tự động gửi một request POST kèm dữ liệu sang URL của ta khi có sự kiện.

Có sự tương đồng với Event-Driven design pattern.

Được ứng dụng rộng rãi trên Github, Discord, ...

```
@app.route("/books", methods=["POST"])
def create_book():
    ...
    dispatch(webhooks, "book.created", book)
    return jsonify(book), 201

def dispatch(webhooks, event_type, data):
    for wh in webhooks:
        if wh["event"] != event_type:
            continue
        Thread(target=_send, args=(wh["url"],
event_type, data), daemon=True).start()
```


Lựa chọn **Giao thức Mạng**

Bên cạnh Design Pattern, việc chọn công nghệ truyền tải quyết định hiệu suất cốt lõi của kiến trúc.

REST API – Resource Based

Giao tiếp dựa trên HTTP method. Dữ liệu trả về được quy định bởi API

gRPC – Action Based

Framework gọi hàm từ xa của Google. Dữ liệu nhị phân siêu nhẹ, tốc độ truyền tải cao.

GraphQL – Query Based

Ngôn ngữ truy vấn phát triển bởi Facebook. Client có quyền tự chỉ định cấu trúc dữ liệu muốn lấy.

| REST

Đặc điểm

- Dữ liệu JSON thô, dễ đọc (Human Readable).
- Cấu trúc được Server định nghĩa cứng (Fixed).
- Dễ bị **Over-fetching/Under-fetching**: Trả về cả những thông tin client không cần dùng.



```
{ "id": 1, "title": "Clean Code", "author": "Robert  
Martin", "pages": 464, "stock": 25, "created_at":  
"2024-01-01" }
```

| gRPC

Đặc điểm

- Truyền tải dưới dạng **nhị phân**.
- Cần file .proto để định nghĩa cấu trúc.
- Hiệu năng cực cao, tiết kiệm băng thông so với JSON.

```
service BookService {  
    rpc GetBookDetails (BookRequest) returns  
    (BookResponse);  
}  
  
message BookRequest {  
    int32 book_id = 1;  
}  
  
message BookResponse {  
    int32 id = 1;  
    string title = 2;  
    string author = 3;  
    int32 pages = 4;  
}
```

GraphQL

Client Query

```
query { book(id: 1) { title author } }
```

Server Response

```
{ "data": { "book": { "title": "Clean Code", "author":  
  "Robert Martin" } } }
```

Response map chính xác theo Query – Không Over-fetching!

So Sánh

Đặc điểm	REST (Representational State Transfer)	gRPC (Google Remote Procedure Call)	GraphQL (Graph Query Language)
Định dạng dữ liệu	Chủ yếu là JSON, XML	Protobuf (Binary)	JSON
Mô hình tương tác	Tài nguyên (Resource-based)	Hành động (Action-based)	Truy vấn (Query-based)
Hiệu năng	Trung bình	Rất cao (Tối ưu cho microservices)	Tốt (Giảm thiểu Over-fetching)
Độ khó	Thấp (Dễ học nhất)	Cao (Cần cài đặt phức tạp)	Trung bình

Khi nào nên dùng?



REST API

Là tiêu chuẩn phổ biến nhất, tập trung vào tính đơn giản và khả năng tương thích cao với trình duyệt. Nó phù hợp nhất cho các API công khai (Public APIs) và các ứng dụng web đơn giản.



gRPC

Được thiết kế để tối ưu hóa hiệu suất với kích thước gói tin nhỏ (binary) và khả năng truyền dữ liệu hai chiều. Đây là lựa chọn hàng đầu cho giao tiếp nội bộ giữa các microservices.



GraphQL

Cho phép client yêu cầu chính xác những dữ liệu họ cần, tránh việc lấy thừa hoặc thiếu dữ liệu (over-fetching/under-fetching). Cực kỳ hiệu quả cho ứng dụng di động hoặc các hệ thống có UI phức tạp.